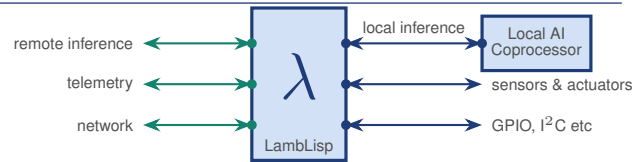


# LambLisp

*From AI to Actuator™*



## A Lisp Language System for the Full Control Stack

LambLisp is a standards-conforming Lisp language system for the full control and AI stack. The same unmodified source code runs on a Linux cloud server doing AI reasoning, an Nvidia Jetson Orin Nano running local neural inference, and an ESP32 microcontroller executing real-time control loops. Every layer exchanges data in the same format as the code itself, so nothing is translated or marshalled between them.

LambLisp nodes communicate with supervisory and cloud tiers over any duplex channel (TCP/IP, UART, etc) for remote evaluation, hot procedure reload, and telemetry — without restarting the device.

LambLisp can also orchestrate the Python and CUDA AI stacks, so the same language that operates the hardware also decides when to run inference and what to do with the result.

|                   |                                                                               |
|-------------------|-------------------------------------------------------------------------------|
| Language standard | Scheme R7RS with real-time enhancements                                       |
| Execution         | Selectable: AST traversal, bytecode VM, or optimized native code              |
| GC                | Adaptive real-time incremental mark-sweep                                     |
| Numeric tower     | Exact integer to arbitrary precision, exact rational, IEEE 754 float, complex |
| Filesystem        | SPIFFS, LittleFS, POSIX                                                       |
| Protocols         | I²C, SPI, WiFi, LLIP; industrial fieldbus <sup>4</sup>                        |
| Platforms         | ESP32-S3, ESP32-C5, ESP32; Nvidia Jetson Orin Nano; Linux x86-64, AArch64     |
| C++ API           | Lamb class; single standard function signature, no FFI required               |

### Scalable Execution Engine

#### AST Traversal

- Evaluates the parsed AST directly; no code generation phase
- Fast pointer threading: direct dispatch to compiled C++ procedures, no boundary overhead
- Tree-walking evaluator with trampoline for tail calls
- Proper tail recursion per R5RS §3.5
- Full debug information retained — names, source structure, expression context

#### Bytecode VM (BC)

- Extends AST traversal with custom bytecode and inline primop expansion
- Compiles on first call; no separate build step or toolchain
- Measured speedup over AST: see *Optimizing Compiler Performance*

#### Native Code Generator (NCG)

- Smart JIT compiler: activates automatically based on heuristic
- Direct NCG→NCG dispatch; no VM re-entry
- Backends: x86-64, AArch64 (Jetson), Xtensa (ESP32, ESP32-S3)
- IRAM placement for hot paths on Xtensa
- AST, BC, and NCG closures interoperate transparently

### Real-Time Dynamic Memory

LambLisp provides a **firm upper bound on garbage-collection activity**. Collection is interleaved with the application in bounded time slices, so the work done in any one slice is capped by construction — and the cap does not grow as the heap fills. Measured on an ESP32-S3 (N8R2): worst-case pause 112–122 μs and median 2–4 μs across heap occupancies from 20% to 50%. A stop-the-world collector's pause scales with the live set; this one does not.

The collector is a real-time adaptive incremental design, extending the work of Dijkstra et al. (1978)<sup>1</sup> and Yuasa (1986)<sup>2</sup> with tunable time slices, adaptive collection triggers, and idle-time amortization.

Triggers and heap floor are recomputed from the live-cell count after every cycle, and idle-time amortization shifts collection away from peak demand — holding loop times uniform and low-jitter.

### Security

LLIP transport is secured with RSA-2048 key exchange, Diffie-Hellman ephemeral key agreement, and AES-GCM authenticated encryption. All cryptographic operations use bignum arithmetic implemented in LambLisp's numeric tower.

On the ESP32, AES and SHA operations use the on-chip hardware acceleration engines, reducing cipher overhead and freeing the application core for control work.

Scheme's minimal, formally-specified grammar also reduces exposure to Trojan-source attacks and makes programs more amenable to automated formal analysis than equivalent C++ or embedded Python.

### Scheme R7RS Compliance

|          |                                                        |                |
|----------|--------------------------------------------------------|----------------|
| §3.5     | Proper tail recursion                                  | ✓              |
| §4.1–4.2 | Core syntax and derived forms                          | ✓              |
| §4.3     | Hygienic macros ( <code>syntax-rules</code> )          | ✓              |
| §6.1     | Equivalence predicates                                 | ✓              |
| §6.2     | Numeric tower                                          | ✓              |
| §6.3–6.8 | Booleans, pairs, symbols, chars, strings, vectors      | ✓              |
| §6.9     | Bytevectors                                            | ✓              |
| §6.10    | Control features                                       | ✓ <sup>3</sup> |
| §6.11    | Exceptions ( <code>guard</code> , <code>raise</code> ) | ✓              |
| §6.13    | I/O                                                    | ✓              |

### Real-Time and AI Extensions

|                                                       |                |
|-------------------------------------------------------|----------------|
| I <sup>2</sup> C, SPI, PWM, GPIO (embedded)           | ✓              |
| WiFi, TCP/IP, UDP                                     | ✓              |
| CUDA inference integration (Jetson / x86-64)          | ✓              |
| Interrupt-driven GPIO                                 | ✓              |
| Modbus RTU / TCP, client and peripheral               | ✓              |
| PROFIBUS DP- and PROFINET-compatible                  | ✓ <sup>4</sup> |
| Typed vectors (n-dimensional arrays)                  | ✓              |
| Heap auto-persistence at boot                         | ✓              |
| Serial REPL over USB / UART (embedded)                | ✓              |
| Native CRC-32 over bytevectors ( <code>crc32</code> ) | ✓              |

### Numeric Tower

*Numbers are promoted or demoted automatically to the most compact and accurate type that preserves value.*

|                |                                                             |
|----------------|-------------------------------------------------------------|
| Exact integer  | 32- and 64-bit exact; promoted automatically as values grow |
| Bignum         | Arbitrary-precision exact; used for RSA and DH cryptography |
| Exact rational | Exact numerator/denominator, reduced                        |
| Floating point | IEEE 754 single and double                                  |
| Complex        | Optimized numeric components                                |

### LLIP — LambLisp Interaction Protocol

LambLisp Interaction Protocol (LLIP) over TCP/IP:

- Remote `eval` with result streaming
- Hot-reload: redefine procedures without restart
- Mutual authentication via RSA-2048
- Session encryption via DH + AES-GCM

### Optimizing Compiler Performance

21 benchmarks tested on ESP32.

|                           | median | best |
|---------------------------|--------|------|
| Bytecode over AST         | 5×     | 31×  |
| Native code over AST      | 16×    | 162× |
| Native code over bytecode | 3×     | —    |

### Tested Platforms

ESP32-S3 (N8R2, N32R16, S3-EYE); ESP32-C5 N8R8 (RISC-V RV32); ESP32 WROVER and 4WD (LX6); Jetson Orin Nano (AArch64, CUDA); Linux and Windows x86-64; WebAssembly.

### Evaluating LambLisp

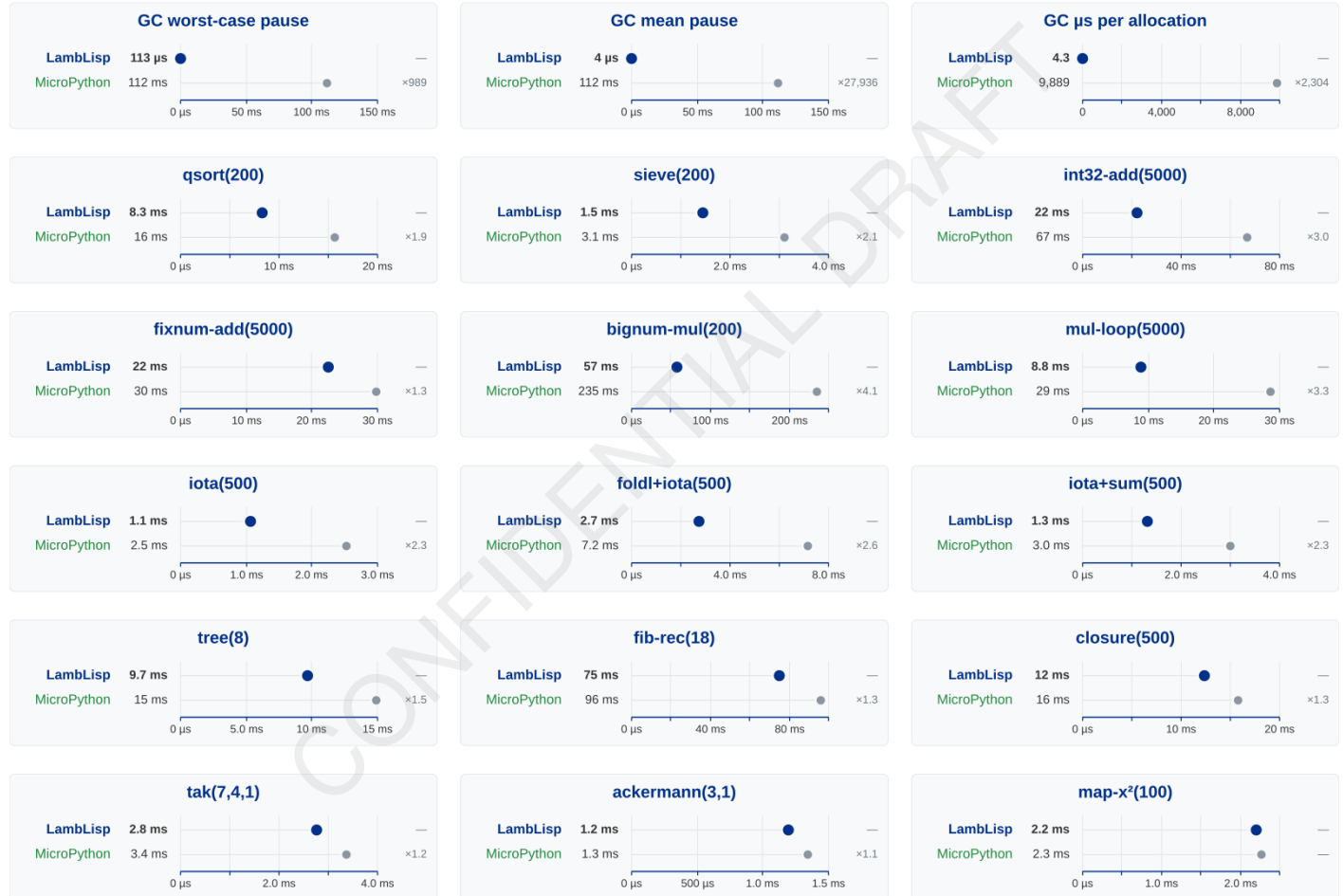
Flash the prebuilt image, open a serial terminal at 115 200 baud, and the `LL=>` prompt is there. No toolchain, no build step, no reflash to change code.

### References

1. Dijkstra, E.W. et al., “On-the-fly Garbage Collection: An Exercise in Cooperation,” *Commun. ACM* 21(11):966–975, 1978.
2. Yuasa, T., “Real-Time Garbage Collection on General-Purpose Machines,” RIMS TR-86-17, Kyoto University, 1986; *J. Syst. Softw.* 11(3):171–179, 1990.
3. `call-with-current-continuation` is escape-only to improve performance; continuations are not re-entrant.
4. Designed to standard IEC 61158/61784 Type 3; PROFINET DCP. PROFIBUS® and PROFINET® are registered trademarks of PROFIBUS Nutzerorganisation e.V. (PI).

## GC Pause Latency and Execution Speed

LambLisp's incremental GC bounds the per-allocation pause to one mark or sweep quantum, independent of live heap size. The stop-the-world collectors must scan the entire live set on every collection, so their worst-case pause grows with the heap.



### Test Conditions

ESP32-S3 rev v0.2 · 2 MB PSRAM · dual-core Xtensa LX7 · 240 MHz

#### GC Test Conditions

Common protocol: 30,000-object live set and 30 collections per runtime, on the same board.

LambLisp: gc-pause-bench; 41,618 of 122,880 cells live (34%); 61,440-cell blocks  $\times 2$ , PSRAM; incremental — 1,346,730 allocs

MicroPython: v1.29.0 (20260824); live set of small tuples; forced full gc.collect() — 18,531 allocs

#### Performance Test Conditions

LambLisp: NCG; PSRAM cell heap; ESP32-S3 dual-core LX7 240 MHz

MicroPython: v1.29.0 (20260824); SPIRAM enabled

## Target Applications

1. **Industrial controls** — Distributed sensor/actuator nodes report to a SCADA-level LambLisp supervisor via LLIP. Updated control logic is pushed live as new code — no firmware update or node reboot required. Continuous telemetry enables automated monitoring and early fault detection.
2. **Medical and assistive devices** — LambLisp's bounded GC pause is a fit for devices with hard timing requirements: infusion pumps, powered mobility aids, and environmental sensors. Treatment protocols and device policies are updated from a clinical or fleet supervisor via LLIP without reflashing.
3. **Autonomy and cooperative behavior** — LambLisp nodes receive motion instructions as executable code — waypoints, tool paths, formation commands — and translate them directly into hardware control. A Jetson edge node performs SLAM or mission planning via CUDA and pushes updated instructions to mobile nodes via LLIP at any time. Applies to multi-axis arms, mobile robots, and aerial and underwater vehicles.
4. **Precision agriculture** — LambLisp sensor nodes in the field report to a Jetson running crop-management logic. CUDA-accelerated imaging drives irrigation and treatment decisions; updated schedules are pushed to actuator nodes as new code.
5. **Aerospace** — Attitude control within a bounded GC pause; maneuver strategies uploaded as new LambLisp expressions via LLIP uplink. Guidance algorithms promoted to native code for deterministic timing; destination and engagement parameters updated as new instructions via encrypted LLIP before or during flight.

## Licensing and Availability

Free for non-commercial use. Commercial use requires a license.

Contact: Clive Smith

[clive.smith@frobeniusnorm.com](mailto:clive.smith@frobeniusnorm.com)

+1 978 566 2300

*Full documentation is available at [lamblisp.com](http://lamblisp.com)*

## Frobenius Norm LLC

[clive.smith@frobeniusnorm.com](mailto:clive.smith@frobeniusnorm.com)

[www.frobeniusnorm.com](http://www.frobeniusnorm.com)

+1 978 566 2300